

Code review — academy-hub-api

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 5 — 2026-09-30 — sha bda6ec9

Scop: `@RestControllerAdvice` — cerut de patru runde, acum scris — plus starea celor 8  din runda 4.

Proiectul **compilează** (`mvn -DskipTests compile` ) și, pentru prima dată în trei runde, **pornește pe o bază de date complet nouă** (2,1 secunde, `GET /api/students` → 200). Toate constatările de mai jos au fost reproduse prin rulare: copie în afara repo-ului, H2 în loc de MySQL, portul 8095, **peste 50 de cereri HTTP reale** plus trei porniri succesive pe o bază H2 pe fișier (ca să pot proba B8 pe date care persistă). Codul din repo nu a fost modificat.

Ce s-a închis

- `@RestControllerAdvice` **EXISTĂ** (cerut la runde 1, 2, 3 și 4) — `GlobalExceptionHandler` cu 11 handleri și `ApiError` ca formă unică de răspuns. Din cele 20 de situații pe care le-am probat, **14 răspund acum corect**, cu status și mesaj. Asta e cea mai mare mișcare de la începutul proiectului și e făcută bine: `ApiError` are `details`, validarea de corp întoarce lista de câmpuri sortată, iar `build()` e o singură metodă privată, nu copy-paste în 11 locuri. Măsurat, printre altele: `POST /api/students` cu 3 câmpuri invalide → **400** cu toate trei în `details`; `GET /api/students/nu-e-uuid` → **400**; `GET /api/nu-exista` → **404**; `POST` cu JSON stricat → **400**; `GET /api/students/search` fără `email` → **400** cu textul „Required query parameter 'email' is missing”.
- `unique = true` **scos din @JoinColumn** (runda 4, B1) — **închis**. Probat exact cum am spus că se probează: bază ștearsă, pornire de la zero, aplicația urcă și seed-ul intră.
- `PUT /api/books/{id}` cu `createdAt` în viitor → **400** cu `"createdAt: Created at cannot be in the future"` (era 500 la runda 4).
- `@Transactional` **pe toate cele patru command service-uri** (runda 4, M1 · runda 3, M9 · runda 2, M6 — deschis de trei runde) — **închis**, pus la nivel de clasă.
- `BookQueryServiceImpl` e `@Service`, nu `@Component` (runda 4, C5) — **închis**.
- Constructorul lui `EnrollmentCommandServiceImpl` **nu mai primește doi repository nefolosiți** (runda 4, C4) — **închis**.
- `GET /{id}` **pe toate patru resursele** + `GET /api/students/search?email=` (runda 4, M9 — parțial). `GET /api/students/{id}` nu mai e 405, iar căutarea după email merge: 200 pe email existent, **404** pe inexistent.
- **Colecția Postman a crescut de la 12 la 28 de cereri**, cu drumuri de eroare printre ele (`GET /api/books/1`, `GET /api/nu-exista`, `search` fără parametru). Ai început să-ți probezi propriile erori — vezi însă B1: ai probat exact cele care merg.

Critice

B1 — Advice-ul nu-și cunoaște jumătate din excepții: 20 de locuri încă dau 500

`web/GlobalExceptionHandler.java:30-31` și `:52`

CUM E ACUM: ai 16 clase de excepție de domeniu. Advice-ul înregistrează **7**. Șapte dintre cele neînregistrate sunt aruncate efectiv, în **20 de locuri**. Reprodus azi:

Cerere	Ce ar trebui	Ce primești	Excepția care iese
GET /api/courses/{id} inexistent	404	500	NoCourseFound
GET /api/courses pe bază goală	200 []	500	NoCourseFound
GET /api/students pe bază goală	200 []	500	NoStudentsFound
GET /api/books pe bază goală	200 []	500	NoBookFound
GET /api/enrollments pe bază goală	200 []	500	NoEnrollmentsFound
POST /api/courses cu nume duplicat	409	500	CourseNameAlreadyInUse
POST /api/enrollments pereche existentă	409	500	StudentAlreadyEnrolled...
POST /api/enrollments cu courseId inexistent	404	500	NoCourseFound
POST /api/books carte „deja atribuită”	409	500	BookAlreadyAssigned...
POST /api/books cu studentId inexistent	404	500	NoSuchElementException
PUT /api/enrollments/{id} aceeași pereche	200	500	StudentAlreadyEnrolled...

Numărătoarea exactă, ca să știi cât ai de legat:

NEÎNREGISTRATE, dar aruncate:

NoStudentsFound	6 locuri
NoCourseFound	6 locuri
NoBookFound	3 locuri
StudentAlreadyEnrolledInThisCourse	2 locuri
NoEnrollmentsFound	1 loc
CourseNameAlreadyInUse	1 loc
BookAlreadyAssignedToThisStudent	1 loc
	20 locuri

ÎNREGISTRATE, dar niciodată aruncate:

EmailNotFound	0 locuri
---------------	----------

DE CE: un `@RestControllerAdvice` nu e o plasă care prinde tot ce trece — e un **registru**, și singura cheie din registru e tipul excepției. La pornire, Spring citește fiecare `@ExceptionHandler` și construiește o hartă *clasă de excepție* → *metodă*. Când o excepție iese din controller, `ExceptionHandlerExceptionResolver` caută în hartă tipul ei, apoi superclasele ei. `NoCourseFound` *extends* `RuntimeException`; în hartă nu există nici `NoCourseFound`, nici `RuntimeException`, deci nu se găsește nimic — și de acolo încolo drumul e exact cel de la runda 4: nimeni nu revendică excepția, cererea e redirectată intern spre `/error`, `BasicErrorController` scrie 500.

Atenție la ce te-a păcălit: handlerul tău de not-found are parametrul declarat `RuntimeException exception`. Asta arată **ca și cum** ar prinde orice `RuntimeException` — dar tipul parametrului nu decide nimic; decid cele șase clase din `@ExceptionHandler({...})`. Parametrul e doar cutia în care Spring îți pune excepția *după* ce a hotărât, din listă, că te cheamă pe tine. Lista e contractul, semnătura nu.

Și uită-te la ce spune asta despre `NoCourseFound` vs `CourseIdNotFound`: ai două excepții pentru „cursul nu există”, iar în registru ai pus-o pe cea greșită. De-aia `PUT /api/courses/{id}` inexistent dă **404** corect (`CourseIdNotFound`, înregistrată), iar `GET /api/courses/{id}` inexistent — scris în același commit cu advice-ul — dă **500** (`NoCourseFound`, neînregistrată). Aceași resursă, același „nu există”, două rezultate.

Cum se prinde asta singur: nu prin citire. Tu ai probat advice-ul cu Postman și ai adăugat în colecție exact cererile care ies bine — nicio cerere cu nume duplicat, nicio pereche retrimisă, nicio listă golită. Un advice se probează pe rândurile care *ar trebui* să fie 409 și 404, nu pe cele care erau deja 200. Regula practică: pentru fiecare `throw new` din `src/`, o cerere care îl declanșează.

FIX: grupează pe familie, o dată, complet:

```
@ExceptionHandler({StudentNotFound.class, StudentIdNotFound.class,
    NoCourseFound.class, CourseIdNotFound.class, BookNotFound.class,
    EnrollmentNotFound.class})
public ResponseEntity<ApiError> handleNotFound(...) { ... 404 ... }

@ExceptionHandler({EmailAlreadyUsed.class, CourseNameAlreadyInUse.class,
    StudentAlreadyEnrolledInThisCourse.class,
    BookAlreadyAssignedToThisStudent.class})
public ResponseEntity<ApiError> handleConflict(...) { ... 409 ... }
```

`NoStudentsFound`, `NoBookFound`, `NoEnrollmentsFound` **nu se înregistrează** — ele dispar odată cu B4, pentru că o listă goală nu e o eroare. `EmailNotFound` și `BookAlreadyExists` nu sunt aruncate de nimeni: șterse.

B2 — REGRESIE: `PUT /api/students/{id}` nu mai poate salva nimic

`student/services/StudentCommandServiceImpl.java:70-72`

```
if(studentRepository.existsByEmail(studentUpdate.email())) {
    throw new EmailAlreadyUsed();
}
```

CUM E ACUM: trei cereri, în ordinea în care le-am trimis, pe același student —

```
PUT /students/{id} cu ACELASI email, varsta 32 -> 409 "Email already used"
PUT /students/{id} cu emailul altui student -> 409 "Email already used"
PUT /students/{id} cu un email NOU -> 200
```

Prima linie e bug-ul. Vreau să-i schimb vârsta studentului și îi trimit datele lui așa cum sunt, cu emailul lui — și primesc „emailul e deja folosit”. **Singurul PUT care trece este cel care schimbă emailul.** Corectarea unei greșeli de tipar în nume, schimbarea vârstei, orice altceva: imposibil prin API.

DE CE: verificarea e copiată din `addStudent ( :34 )`, unde e corectă, și mutată într-o situație în care întrebarea nu mai e aceeași. La creare, studentul nu există încă: dacă emailul apare în bază, apare obligatoriu la **altcineva**. La update, studentul există deja și **emailul lui e în bază, pe rândul lui**. `existsByEmail` întoarce `true` din cauza propriului rând. Rândul pe care îl modifici se găsește pe sine și se declară propriul duplicat — e, literă cu literă, același mecanism ca la B7 de la înscrieri, care e deschis de trei runde.

Partea care contează pentru runda următoare: fixul ăsta **era scris** în review-ul rundei 4, la B6, cu cod, iar sub el era o secțiune numită „Atenție la forma verificării”, care spunea că `existsByEmail` copiat din `addStudent` ar refuza orice `PUT` care nu schimbă emailul, și că e exact capcana de la B7. Ai luat regula („verifică unicitatea și la update”) și ai lăsat forma. De-aia 409-ul de la linia 2 arată ca un succes: statusul e cel cerut, din motivul greșit — verificarea nu s-a uitat niciodată la *cine* deține emailul.

O întrebare de unicitate la update are întotdeauna două jumătăți: „există valoarea?” și „la altcineva decât la mine?”. A doua jumătate nu e un detaliu de stil; fără ea, verificarea nu compară două entități, compară o entitate cu ea însăși.

FIX:

```
studentRepository.findByEmail(studentUpdate.email())
    .filter(other -> !other.getId().equals(id))
    .ifPresent(other -> { throw new EmailAlreadyUsed(); });
```

`findByEmail` există deja — l-ai adăugat în `StudentRepository:51` în același commit.

Cum verifici că ai reparat: `PUT` cu emailul nemodificat → 200. `PUT` cu emailul altui student → 409. Amândouă, nu doar a doua.

B3 — GET /api/enrollments/{id} inexistent → 500 prin NullPointerException

`enrollment/services/EnrollmentQueryServiceImpl.java:38` · `enrollment/repository/EnrollmentRepository.java:15`

```
public EnrollmentResponse getEnrollmentById(UUID id) {
    return EnrollmentResponse.from(enrollmentRepository.getById(id));
}
```

CUM E ACUM:

```
GET /api/enrollments/{id valid}      -> 200
GET /api/enrollments/0000...0000     -> 500
```

Celelalte trei `GET /{id}`, scrise în același commit, folosesc `findById(...).orElseThrow(...)` și dau 404 sau 500 *din cauza registrului* (B1). ăsta e singurul care crapă altfel.

DE CE: `Enrollment getById(UUID id)` din `EnrollmentRepository` e o **derived query** scrisă de tine — Spring Data îi construiește implementarea din nume, ca la `findByStudentId`. Iar o derived query care întoarce tipul entității, nu `Optional`, are o singură cale să spună „n-am găsit”: **întoarce null**. Nu aruncă nimic. Deci `getById` îți dă `null`, `null` intră în `EnrollmentResponse.from(...)`, iar acolo prima linie este `enrollment.getId()` → `NullPointerException`.

De-aia eroarea arată diferit de toate celelalte: nu e excepția ta de domeniu care nu găsește un handler, e o excepție tehnică apărută **în interiorul** metodei, dintr-un `null` pe care codul tău l-a acceptat ca dacă ar fi o entitate. `Optional` există exact ca să

nu poți face greșeala asta: `findById` te obligă să spui ce se întâmplă când lipsește, `getById` te lasă să uiți.

Încă un motiv să nu folosești numele ăsta: `getById` era o metodă a lui `JpaRepository`, scoasă din uz fix pentru că oamenii credeau că întoarce entitatea, când ea întorcea un proxy leneș. Numele e minat.

FIX: aceeași formă ca la celelalte trei, și șterge `getById` din repository:

```
return EnrollmentResponse.from(enrollmentRepository.findById(id)
    .orElseThrow(EnrollmentNotFound::new));
```

B4 — Listele goale răspund tot 500 — a treia rundă — și direcția s-a inversat în 5 metode

`student/services/StudentQueryServiceImpl.java:29, :43, :50, :58, :98, :105, :112, :124` ·
`course/services/CourseQueryServiceImpl.java:36, :47, :65, :85` · `book/services/BookQueryServiceImpl.java:32, :38, :52` · `enrollment/services/EnrollmentQueryServiceImpl.java:30`

CUM E ACUM: am șters prin API toți studenții, apoi toate cursurile. Toate cele patru liste, reprodus azi:

```
GET /api/students      pe baza goala -> 500
GET /api/courses       pe baza goala -> 500
GET /api/books         pe baza goala -> 500
GET /api/enrollments   pe baza goala -> 500
```

Partea nouă e mai importantă decât repetarea. În `dfa7252` ai atins cinci verificări de listă — dar le-ai împins în **direcția opusă**:

```
ÎNAINTE: if(studentRepository.findAllByAgeGreaterThan(age) == null)
DUPĂ:    if(studentRepository.findAllByAgeGreaterThan(age).isEmpty())
```

`StudentQueryServiceImpl:43, :50, :58` și `BookQueryServiceImpl:38, :52`.

DE CE: cele cinci linii erau semnalate la runda 4 la **M4** — „verificări `== null` care nu se pot declanșa niciodată” — pentru că o metodă derivată care întoarce `List` întoarce o listă goală, nu `null`. Constatarea M4 spunea că `if`-ul e mort. Tu l-ai făcut viu.

Numai că motivul pentru care `if`-ul era o problemă nu era „nu se declanșează” în sine — era că **nu trebuia să existe**: exact asta scria B8 în aceeași rundă, cu fixul „scoți `throw` din cele 13 metode de listare și returnezi lista așa cum vine”. Cele două constatări vorbeau despre aceleași linii din două direcții, iar tu ai citit-o pe cea din care rezulta mai puțin de scris. Rezultatul: cinci metode care înainte tăceau, acum aruncă.

Practic nu se vede încă, pentru că cele cinci nu sunt expuse prin HTTP. Se vede în momentul în care le expui — și fișa **ENDPOINTURI-LIPSA** îți cere fix asta.

Diferența, încă o dată, cu cuvintele tale din cod: la `getBookById` ai scris `orElseThrow(BookNotFound::new)` — **corect**, pentru că acolo ai numit un lucru anume și el lipsește. La `getBooks()` ai scris `if (books.isEmpty()) throw` — **greșit**, pentru că nu ai numit nimic. O listă nu poate lipsi; poate doar să fie goală. Distincția o aplici corect de o rundă la `findById`; e neaplicată exclusiv pe liste.

FIX: scoți `throw`-ul din cele 16 metode de listare (nu doar din cele 5 pe care le-ai atins) și întorci lista. `NoStudentsFound`, `NoCourseFound` pe liste, `NoBookFound`, `NoEnrollmentsFound` rămân nefolosite pe drumul de listare și se șterg.

B5 — `orElseThrow()` fără argument: tot 7 locuri, neatins (runda 4, B5)

`book/services/BookCommandServiceImpl.java:45` , `:69` · `enrollment/services/EnrollmentCommandServiceImpl.java:69` ,
`:83` , `:84` , `:85` · `student/services/StudentQueryServiceImpl.java:69`

CUM E ACUM:

```
POST /api/books {"name":"Ceva","studentId":"0000...0000"} -> 500
```

DE CE: e explicația de la runda 4, pe care o repet pentru că niciuna din cele șapte linii n-a fost atinsă — dar acum are o consecință care la runda 4 era doar prezisă. Atunci am scris: „Diferența nu se vede acum, pentru că fără advice ambele ies 500 — dar în ziua în care scrii advice-ul, cele cu `orElseThrow()` gol vor rămâne singurele care răspund tot 500.”

Ziua aceea a venit. `orElseThrow()` fără argument aruncă `NoSuchElementException`, o excepție din JDK care nu e și nu poate fi în registrul tău de domeniu. Acum că `POST /api/books` cu un `studentId` greșit e singurul „nu există” din tot API-ul care nu dă 404, ai dovada pe masă: excepția tehnică nu are cum să devină un răspuns bun, pentru că nu spune despre ce e vorba.

`StudentNotFound` e importat în `BookCommandServiceImpl`, la linia 10. Tot nefolosit, a doua rundă.

FIX: fiecare `orElseThrow()` primește excepția lui de domeniu — șapte locuri, în același pas, nu pe cel semnalat.

B6 — Verificarea de duplicat la carte, neatinsă (runda 4, B4)

`book/services/BookCommandServiceImpl.java:38–41`

```
if(!bookRepository.findByStudentId(bookCreateRequest.studentId())
    .isEmpty()
    && bookRepository.existsByName(bookCreateRequest.name())) {
    throw new BookAlreadyAssignedToThisStudent();
}
```

CUM E ACUM: reprodus din nou, identic cu runda 4. Maria avea `Head First Java` și `Learning SQL`; `Effective Java` e la Rareș, nu la ea.

```
POST /books {Refactoring, Maria} -> 200   corect, carte noua
POST /books {Effective Java, Maria} -> 500   GRESIT, Maria nu are cartea
```

DE CE: cele două condiții nu vorbesc despre același lucru. Prima întreabă „studentul ăsta are **vreo** carte?”, a doua „titlul ăsta există la **cineva**, oriunde?”. Din ele nu rezultă întrebarea pe care voiai s-o pui: „studentul ăsta are **cartea asta**?”. Un `&&` între două întrebări despre mulțimi diferite nu e o verificare mai strictă, e o verificare despre altceva.

E, în fond, aceeași greșeală ca la B2, cu semnul invers: acolo verificarea nu se uita la *cine* deține valoarea, aici nu se uită la *care* valoare e a cui. Ambele apar când condiția e compusă din bucăți disponibile, în loc să fie scrisă din întrebare.

FIX: `existsByStudentIdAndName(studentId, name)` în `BookRepository` — Spring Data ți-o generează din nume — și `if` pe rezultatul ei. O întrebare, despre pereche, exact ca `existsByStudentIdAndCourseId` pe care o folosești deja la înscrieri (`:39`).

B7 — `PUT` pe o înscriere cu aceeași pereche → 500 — a treia rundă, linia neatinsă

`enrollment/services/EnrollmentCommandServiceImpl.java:80`

```
if(enrollmentRepository.findByStudentIdAndCourseId(eur.studentId(),
    eur.courseId()).isPresent()) {
    throw new StudentAlreadyEnrolledInThisCourse();
}
```

CUM E ACUM: reprodus azi, a treia rundă la rând:

```
PUT /api/enrollments/{id} cu perechea pe care o are deja -> 500
```

DE CE: rândul pe care îl modifici se găsește pe sine în căutare și se declară propriul duplicat. E **exact** mecanismul de la B2, în celălalt modul: `findByStudentIdAndCourseId` răspunde la „există perechea?”, dar întrebarea de la update e „există **altă** înscriere cu perechea asta?”.

Că cele două sunt aceeași greșeală se vede și din fix: același `.filter(... -> !...getId().equals(id))`. Le pun una lângă alta pentru că B2 s-a născut din încercarea de a repara B6 de la runda 4 — ai scris a doua oară bug-ul care era deja deschis în fișierul de alături, cu diagnostic și cu fix, de două runde.

FIX:

```
enrollmentRepository
    .findByStudentIdAndCourseId(eur.studentId(), eur.courseId())
    .filter(e -> !e.getId().equals(id))
    .ifPresent(e -> { throw new StudentAlreadyEnrolledInThisCourse(); });
```

B8 — Fiecare pornire șterge tot ce a scris API-ul (runda 4, B2 — neatins)

Runner.java:93-94 · :100-105 · :121-124

CUM E ACUM: ca să-l probez pe date care persistă, am pornit copia pe un H2 **pe fișier**, nu în memorie. Doi pași, fără să ating nimic între ei:

```
pornire 1: 8 studenti din seed
+ POST /api/students ion.pop@test.ro -> 200
+ POST /api/students ana.ene@test.ro -> 200
10 studenti in baza
--- opresc si repornesc aplicatia, aceeasi baza ---
pornire 2: 8 studenti
supravietuitori test.ro: []
```

DE CE: `Runner` e `@Component` și implementează `CommandLineRunner`, deci `run()` nu e un script pe care îl pornești când vrei — e o metodă pe care Spring o cheamă **la fiecare pornire**. Înăuntru, `reset()` face `deleteAllInBatch()` pe toate patru tabelele, iar `seed()` mai face pe deasupra încă un rând de `deleteAll()` (:121-124): ștergerea se întâmplă de două ori.

Cât timp datele intrau tot din `Runner`, nu se vedea — ștergea și rescria exact ce ștersese. De când ai API, datele au **două surse** și numai una supraviețuiește repornirii. Orice restart din IntelliJ, orice `docker compose up` al doilea, orice cădere cu restart automat înseamnă pierderea a tot ce a scris un client.

Și e legat de ce ai livrat săptămâna asta: advice-ul nu se poate proba serios pe o bază care se resetează singură. Jumătate din situațiile din B1 se văd doar pe stări pe care le construiești tu prin cereri — o listă golită, un duplicat creat acum. Cât timp `Runner`

rescrie seed-ul la fiecare pornire, nu ai cum să ajungi la ele.

FIX: `@Profile("seed")` pe `Runner`, pornit explicit doar când vrei date de test. Varianta minimă: comentezi `reset()` și `seed()` înapoi, exact cum erau la runda 3.

B9 — La update pe curs, regula ta de unicitate nu se execută niciodată

`course/services/CourseCommandServiceImpl.java:51-58`

CUM E ACUM: statusul arată bine, corpul arată de ce nu e:

```
PUT /api/courses/{id} cu numele altui curs
-> 409 {"message":"Request violates a database constraint"}
```

409 e statusul corect. Dar mesajul nu e al tău, e al handlerului de `DataIntegrityViolationException`: cererea a trecut prin `updateCourse` fără nicio verificare, a scris numele în entitate, a ajuns la `flush`, iar **baza de date** a refuzat scrierea. `CourseNameAlreadyInUse`, scrisă anume pentru cazul ăsta, nu se execută. `updateCourse` n-a fost atins de la runda 3.

DE CE: `createCourse` întreabă `existsByNameIgnoreCase` (:28); `updateCourse` încarcă cursul și trece direct pe `setName`. Regula a fost scrisă pe drumul pe care l-ai testat — crearea — și n-a fost dusă pe al doilea drum care duce în aceeași coloană unică. E a treia rundă cu constatarea asta (runda 3, M8 · runda 4, B6).

Ce e nou e că acum **advice-ul te ascunde**. Fără el, lipsa verificării se vedea ca un 500 și era evident că ai o gaură. Cu handlerul de `DataIntegrityViolationException`, orice constrângere încălcată în bază devine un 409 frumos, iar o verificare lipsă arată identic cu una prezentă. Plasa de siguranță a devenit motivul pentru care nu mai vezi ce lipsește.

Mai e și partea în care handlerul acela e prea generos: `DataIntegrityViolationException` acoperă și `NOT NULL`, și cheia străină, și lungimea coloanei — lucruri care nu sunt conflicte, ci cereri invalide (400) sau chiar defecte ale tale (500). Un singur `catch` peste toată clasa de erori de integritate spune „e vina clientului” chiar și când nu e.

FIX: verificarea, în `updateCourse`, în forma de la B2:

```
courseRepository.findByNameIgnoreCase(request.name())
    .filter(other -> !other.getId().equals(id))
    .ifPresent(other -> { throw new CourseNameAlreadyInUse(); });
```

Handlerul de `DataIntegrityViolationException` rămâne, dar ca ultimă plasă, nu ca regulă.

🟡 Importante

M1 — `NoCourseFound` și `CourseIdNotFound` sunt același lucru, scris de două ori. Prima aruncată în 6 locuri, a doua în 4, amândouă înseamnă „cursul nu există”, și doar a doua e în advice (de aici jumătate din B1). Alege una, șterge cealaltă. La fel: `StudentNotFound` (7 locuri) vs `StudentIdNotFound` (6 locuri). `EmailNotFound` e înregistrată în advice și nearuncată nicăieri; `BookAlreadyExists` și `TextArgumentNumberInvalid` nu sunt folosite de nimeni.

M2 — `findByDepartmentOrderByNameAsc` tot nu sortează. `CourseQueryServiceImpl.java:61` cheamă `courseRepository.findByDepartment(department)`; derived query-ul cu sortare nu e apelat. Numele metodei promite o ordine pe care nu o livrează (runda 4, M2).

M3 — interogarea rulează de două ori, în 12 locuri. `StudentQueryServiceImpl.java:29+32 , :36+39 , :43+46 , :50+53 , :58+61 , :66+69 , :74+77 , :82+85 , :98+101 , :105+108 , :112+115` · `BookQueryServiceImpl.java:38+41 , :52+55` . Tiparul e „întreb dacă e goală, apoi o cer din nou”: două `SELECT` identice pentru un singur răspuns. Semnalat la runde 2 (M4), 3 (M2) și 4 (M3). Se rezolvă singur odată cu B4: dacă nu mai verifici, nu mai ceri de două ori.

M4 — verificări `== null` care nu se pot declanșa. `EnrollmentCommandServiceImpl.java:43` și `:46` :
`if(studentQueryService.getStudentById(...)==null) throw new StudentIdNotFound();` · `getStudentById` nu întoarce `null` — aruncă. Deci `StudentIdNotFound` din linia 44 nu se execută niciodată, iar 404-ul de la `POST /api/enrollments` cu student inexistent vine, de fapt, din `StudentNotFound` aruncată o linie mai sus. Același lucru la curs — și acolo e și sursa unui 500 din B1. Semnalat la runde 2 (M5), 3 (M3) și 4 (M4).

M5 — `POST` răspunde 200, nu 201, pe toate patru modulele. `StudentController.java:45` · `CourseController.java:39` · `BookController.java:41` · `EnrollmentController.java:38` . Verificat pe antetele răspunsului: `HTTP/1.1 200` , fără `Location` . La creare se întoarce `201 Created` cu `Location` spre resursa nouă — pe care acum o ai, de când există `GET /{id}` . Semnalat la runde 3 (M4) și 4 (M5).

M6 — `DELETE /api/enrollments` ia id-urile din corpul cererii. `EnrollmentController.java:48-52` . Merge (200, rulat azi), dar multe proxy-uri și clienți HTTP aruncă corpul unui `DELETE` . Forma așteptată: `DELETE /api/enrollments/{id}` — și acum că ai `GET /api/enrollments/{id}` , inconsecvența e în același controller, la 20 de linii distanță. Semnalat la runde 3 (M5) și 4 (M6).

M7 — `PUT /api/books/{id}` cere `createdAt` obligatoriu și îl suprascrive. `BookUpdateRequest.java:12` + `BookCommandServiceImpl.java:79` . Data la care un rând a fost creat nu e un câmp editabil: o pune serverul o dată, la creare (runda 4, M7).

M8 — ștergerea unui curs șterge în tăcere înscrierile. Reprodus azi: `DELETE /api/courses/{id}` pe un curs cu 4 înscrieri → 200; înscrierile din bază scad de la 17 la 13, iar răspunsul nu spune nimic despre ele. Vine din `cascade = CascadeType.ALL` + `orphanRemoval = true` pe `Course.enrollments` . Alternativa uzuală: refuzi ștergerea cât timp există înscrieri (409) sau marchezi cursul inactiv (runda 4, M8).

M9 — 19 din 27 de metode de query tot nu sunt chemate de nimeni. Ai expus 8 (4 liste + 4 `GET /{id}`) plus `search?email=` ; restul stau scrise și nefolosite. Fișa [ENDPOINTURI-LIPSA.pdf](#) le ia pe rând, în ordinea în care merită expuse. Atenție când le expui: fiecare metodă de listare de acolo are `throw` -ul de la B4 în ea, deci expunerea fără B4 înseamnă un 500 nou pe fiecare.

M10 — structura cerută la runde 1, 2, 3 și 4, neatinsă. `ExceptionConstants` în patru pachete cu același nume · `services/interfaces/` în loc de `services/` + `services/impl/` · plural inconsecvent: `models` , `dtos` , `services` , `controllers` , dar `repository` la singular. Pachetul nou `web/` e, în schimb, exact unde trebuie.

M11 — trei convenții de nume în același modul. `EnrollmentCreateResponse` întoarce `enrollmentId` , `studentId` , `courseId` ; `EnrollmentDeleteResponse` întoarce `enrollmentID` , `studentID` , `courseID` ; `EnrollmentUpdateResponse` întoarce `id` , `studentId` , `courseId` . Verificat azi pe răspunsuri reale, toate trei. Semnalat la runda 4 (M11). `ApiError` arată cum se face: o formă, aceeași peste tot.

M12 — două adnotări `@Transactional` diferite în același proiect. `StudentCommandServiceImpl` folosește `org.springframework.transaction.annotation.Transactional` , celelalte trei `jakarta.transaction.Transactional` . Se comportă asemănător, dar nu identic (regulile de rollback și atributele diferă). Alege-o pe cea Spring, peste tot. Plus: `StudentCommandServiceImpl` o are și pe clasă (`:21`) și pe fiecare metodă (`:31` , `:55` , `:65`) — a doua e redundantă.

M13 — `@PathVariable` importat într-o interfață de service. `EnrollmentQueryService.java:5` . O adnotare de web MVC nu are ce căuta în stratul de business, nici măcar ca import nefolosit: e semnul că semnătura a fost copiată din controller.

Cleanups

- **C1** — **22 de importuri nefolosite**, în 19 fișiere. Cele mai vizibile: `BookCommandServiceImpl.java:5` (`BookAlreadyExists`), `:10` (`StudentNotFound` — cea pe care B5 o cere) · `StudentCommandServiceImpl.java:6` , `:8` , `:16` · `EnrollmentCommandServiceImpl.java:4` , `:13` (rămase de la constructorul curățat) · `BookQueryServiceImpl.java:9` (`Component` , rămas după trecerea la `@Service`), `:12` · `CourseQueryServiceImpl.java:14` · `BookController.java:5` · `CourseSummary.java:5` · `StudentUpdateRequest.java:5` · `EnrollmentQueryService.java:4` , `:5` . Tiparul e constant: repara o linie, importul vechi rămâne. IntelliJ le arată gri; `Ctrl+Alt+0` le scoate pe toate dintr-un fișier.
 - **C2** — `deletebook` , `updatebook` cu literă mică, în interfață și în implementare. Semnalat la runde 2, 3 și 4.
 - **C3** — `EnrollmentController.java:27` : metoda care întoarce o înscriere se numește `getCourseById` . Copiată din `CourseController` și neredenumită.
 - **C4** — `Runner.java` are 718 linii, majoritatea metode de test comentate. Odată ce ai controllere și 28 de cereri în Postman, locul lor e la gunoi.
 - **C5** — `StudentUpdateRequest.java:19` : `@NotNull` pe un `int` nu se poate declanșa — un primitiv nu e null. Ce te apără e `@Positive` de dedesubt.
 - **C6** — `Student.java:75-81` : constructorul `Student(String text)` care parsează CSV, rămas de la `Runner` . Nimic din API nu-l cheamă.
 - **C7** — `BookQueryServiceImpl.java` se termină fără linie nouă, cu trei rânduri goale înainte de acoladă.
 - **C8** — `springdoc-openapi-starter-webmvc-ui` are versiune fixată în `pom.xml` (`3.1.0`), deși toate celelalte dependențe o iau din `spring-boot-starter-parent` . Funcționează, dar la următorul upgrade de Boot rămâne în urmă singură. Bun că l-ai adăugat: `/swagger-ui.html` îți dă gratis lista de endpointuri de la M9.
-

Before / After (doar pentru 🚫)

#	Acum	Corect
B1	7 din 16 excepții înregistrate; 20 de <code>throw</code> cad pe <code>/error</code>	un <code>@ExceptionHandler</code> per familie, cu TOATE clasele: not-found → 404, conflict → 409
B2	<code>existsByEmail(request.email())</code> la update	<code>findByEmail(...).filter(o -> !o.getId().equals(id)).ifPresent(...)</code>
B3	<code>EnrollmentResponse.from(repo.getById(id))</code>	<code>repo.findById(id).orElseThrow(EnrollmentNotFound::new)</code>
B4	<code>if (lista.isEmpty()) throw new NoStudentsFound();</code>	<code>return lista;</code> — lista goală e <code>200 []</code>
B5	<code>findById(id).orElseThrow()</code>	<code>findById(id).orElseThrow(StudentNotFound::new)</code> — în toate 7
B6	<code>!findByStudentId(sid).isEmpty() && existsByName(name)</code>	<code>existsByStudentIdAndName(sid, name)</code>
B7	<code>findByStudentIdAndCourseId(...).isPresent()</code>	<code>.filter(e -> !e.getId().equals(id)).ifPresent(...)</code>
B8	<code>reset(); seed();</code> la fiecare pornire	<code>@Profile("seed")</code> pe <code>Runner</code>
B9	<code>updateCourse</code> scrie numele fără verificare	<code>findByNameIgnoreCase(...).filter(altul).ifPresent(...)</code>

Q&A — de răspuns înainte de runda 6

1. **B1.** Handlerul tău de not-found are parametrul `RuntimeException exception` și totuși `NoCourseFound` (care e o `RuntimeException`) nu ajunge la el. Ce anume decide, la pornirea aplicației, care excepție ajunge la care metodă — și de ce tipul parametrului nu are nicio treabă cu decizia asta?
2. **B2 + B7 + B9.** Trei module, aceeași greșeală: verificarea de unicitate la update găsește rândul propriu. Scrie în cuvinte de ce `existsByEmail` e **corectă** în `addStudent` și **greșită** în `updateStudent` — ce s-a schimbat între cele două situații în ce privește ce conține baza în momentul verificării?
3. **B4.** La runda 4 aceleași cinci linii apăreau în două constatări: M4 („`if` -ul nu se poate declanșa”) și B8 („scoate `throw` -ul, lista goală nu e eroare”). Tu ai făcut `if` -ul să se declanșeze. Ce întrebare ți-ai fi putut pune despre `getBooks()` — comparând-o cu `getBookById()`, pe care ai scris-o corect — care ar fi arătat că cele două constatări cereau același lucru?

Runda 4 — 2026-09-25 — sha `8ecffc9`

- 🟢 B1 `unique = true` pe ambele `@JoinColumn` → aplicația nu pornește pe bază nouă — **închis** (probat pe bază ștersă, pornire în 2,1s)
- 🚫 B2 `Runner` șterge tot la fiecare pornire — **NEATINS** (runda 5, B8)
- 🟡 B3 niciun `@RestControllerAdvice`, 12 situații → 500 — **închis pe jumătate**: advice-ul există, 14 din 20 de situații răspund corect, 20 de `throw` rămân pe 500 (runda 5, B1)
- 🚫 B4 duplicatul de carte întreabă despre mulțimi diferite — **NEATINS** (runda 5, B6)

- B5 `orElseThrow()` gol în 7 locuri — **NEATINS** (runda 5, B5)
- B6 unicitate lipsă la update pe student și pe curs — „reparat” cu bug nou: pe student a intrat `existsByEmail` și a blocat orice `PUT` (runda 5, B2); pe curs, neatins (runda 5, B9)
- B7 `PUT` pe aceeași pereche de înscriere → 500 — **NEATINS**, a treia rundă (runda 5, B7)
- B8 listele goale → 500, 13 locuri — **NEATINS**, iar 5 verificări moarte au fost activate în loc de șterse (runda 5, B4)
- M1 `@Transactional` lipsă pe command service-uri — **închis** (toate patru)
- M2 `findByDepartmentOrderByNameAsc` nu sortează — **NEATINS** (runda 5, M2)
- M3 query dublu, 12 locuri — **NEATINS** (runda 5, M3)
- M4 `== null` imposibil în `EnrollmentCommandServiceImpl` — **NEATINS** (runda 5, M4)
- M5 `POST` → 200 în loc de 201, 4 locuri — **NEATINS** (runda 5, M5)
- M6 `DELETE /api/enrollments` cu corp — **NEATINS** (runda 5, M6)
- M7 `createdAt` editabil la `PUT /api/books` — **NEATINS** (runda 5, M7)
- M8 ștergerea cursului șterge înscrierile în tăcere — **NEATINS**, reprodus din nou (runda 5, M8)
- M9 niciun `GET /{id}` — **închis**: toate patru resursele + `search?email=` (runda 5, M9 pentru restul metodelor neexpuse)
- M10 structura (`ExceptionConstants` ×4, `services/interfaces/`, `repository`) — **NEATINSĂ**, a patra rundă (runda 5, M10)
- M11 trei convenții de nume în JSON-ul de enrollment — **NEATINS** (runda 5, M11)
- C4 constructor cu repository neatribuți · C5 `@Component` pe `BookQueryServiceImpl` · C1, C2 importuri nefolosite semnalate punctual — **închise**
- C3, C6, C7, C8, C9, C10 — **neatinse** (runda 5, C1–C6)

Runda 3 — 2026-09-23 — sha 62915d4

- B4 `@Valid` fără reguli pe `EnrollmentUpdate/DeleteRequest` — **închis** (`PUT {}` → 400)
- B5 `createBook` nu putea reuși niciodată — **închis** (`POST /api/books` → 200)
- B6 trei command service-uri inaccesibile din API — **închis** (CRUD pe toate patru)
- B1 liste goale → 500 — **NEATINS**, 13 locuri (runda 4, B8)
- B2 niciun `@RestControllerAdvice` — **NEATINS**, acum 12 situații (runda 4, B3)
- B3 `PUT` pe aceeași pereche → 500 — **NEATINS** (runda 4, B7)
- M6 `Enrollment.equals` crăpa fără student/curs — **închis**
- M7 `deleteStudent` ignora emailul — **închis** (semnătura trecută pe `UUID`)
- M2 query dublu — **închis** în `CourseQueryService` , rămas în celelalte (runda 4, M3)
- M8 unicitate lipsă la update — **NEATINS**, acum expus prin HTTP (runda 4, B6)
- M1 graf cu două colecții · M3 `== null` imposibil · M4 `POST` → 200 · M5 `DELETE` cu corp · M9 `@Transactional` lipsă · M10 structura — **toate NEATINSE** (runda 4, M1–M10)
- C1, C2, C3, C4, C5, C7 — **neatinse** (runda 4, C1–C6)

Runda 2 — 2026-08-19 — sha 8b84a6d

- B1 `Course.enrollments` neinițializat → NPE — **închis**
- B2 `StudentSummary.getId()` `Long` peste `UUID` — **închis**
- B3 `@EntityGraph` către un graf inexistent — **închis**
- B4 `createBook` nu poate reuși niciodată — **NEATINS** (runda 3, B5)

- 🟡 B5 `updateEnrollment` duplică perechea — verificarea a fost adăugată, dar prinde și rândul propriu (runda 3, B3);
constrângerea `UNIQUE` pe tabelă tot lipsește
- 🔴 B6 `Enrollment.equals` crapă fără student/curs — **NEATINS** (runda 3, M6)
- 🔴 B7 `deleteStudent` ignoră emailul din request — **NEATINS** (runda 3, M7)
- M1 `updateCourse` scrie orbește · M2 email duplicat la update · M3 excepție greșită · M4 query dublu · M5 `== null` imposibil
· M6 `@Transactional` lipsă · M7 factory · M8 `Enrollment` incomplet — **toate NEATINSE** (runda 3, M2–M9)
- C1–C8 — neatins, cu excepția celor dispărute odată cu refacerea fișierelor

Runda 1 — 2026-08-05

- ✅ `spring-boot-starter-web` + `controllers/` — **închise la runda 3**
- 🔴 `@RestControllerAdvice` — **NEATINS de trei runde** (runda 3, B2)
- 🔴 `services/interfaces/` → `services/` + `services/impl/` — **NEATINS**
- 🔴 `ExceptionConstants` duplicat (acum în 4 pachete) — **NEATINS**
- 🔴 plural inconsistent `repository` vs `models / dtos / services` — **NEATINS**
- Decizia de arhitectură: `Student` = aggregate root, `book/` fără `services/` proprii — contrazisă de `BookCommandService`,
de reluat explicit

